

Workshop

CI for Robots

Github Actions + ROS2

—

Presenter: 吳沛昀 鄒潔瑤

Date: 2026/08/27

Introduction - CI

Nobody did anything wrong

FRI 17:40 Someone pushes a small change to the velocity limiter.

FRI 17:41 It builds fine on their machine — the stale install/ still has the old headers.

MON 09:15 Someone else pulls main. The robot will not build.

MON 12:30 Two people have lost a morning.

That's why we need CI

- CI = Continuous Integration
- automatically check every code change
- Typical checks:
 - install dependencies
 - build
 - run tests
 - code quality (style)
- Goal: find problems early and keep *main* healthy

Introduction - ROS2

Why ROS 2 is our workshop setting

ROS 2 = a framework + ecosystem
for building robot software from modular components

...so we learn CI on a small ROS 2 project :)

Package

A package is the unit ROS builds, tests, installs, and shares.

```
turtle_guard/  
├── package.xml  
├── setup.py  
├── test/  
└── turtle_guard/  
    ├── safety.py  
    └── velocity_guard_node.py
```

package.xml

ROS metadata +
declared dependencies

setup.py

How this Python
package is installed

turtle_guard/

The actual source code

test/

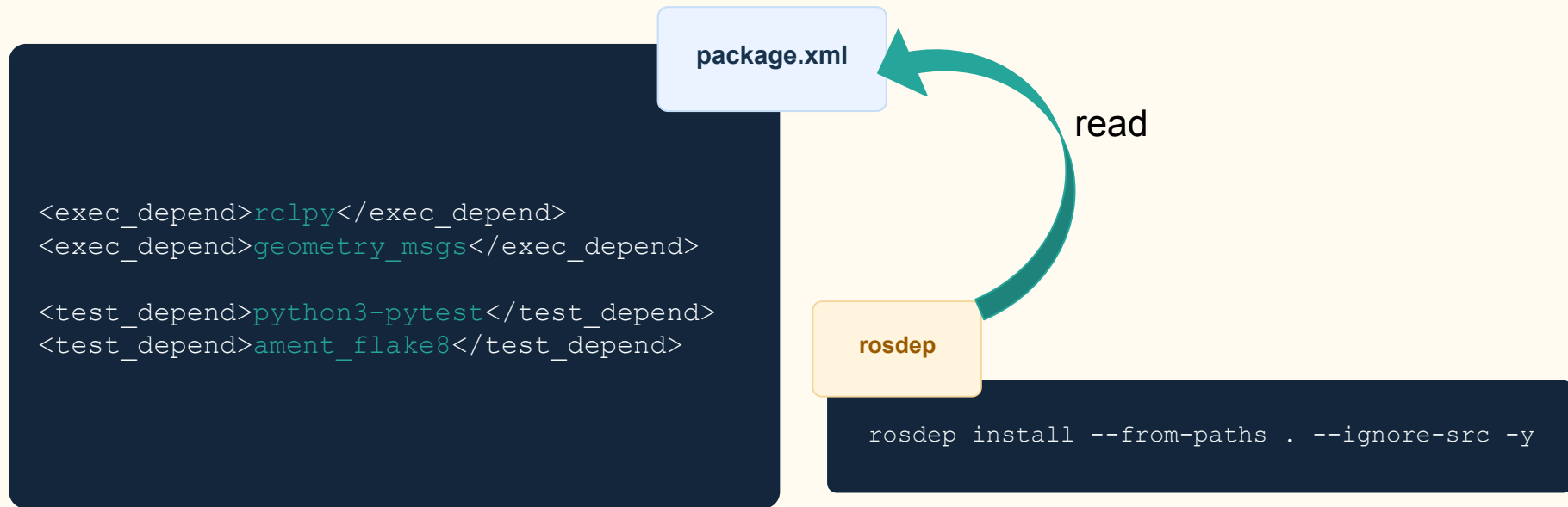
Unit tests + lint tests

```
colcon build --packages-select turtle_guard
```

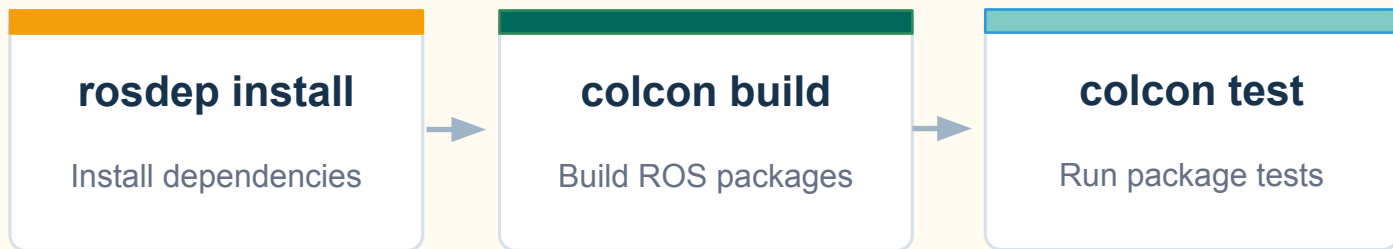
“Build the ROS package named turtle_guard.”

Dependencies: why we need rosdep

Your laptop may already have libraries installed. A CI container starts fresh.

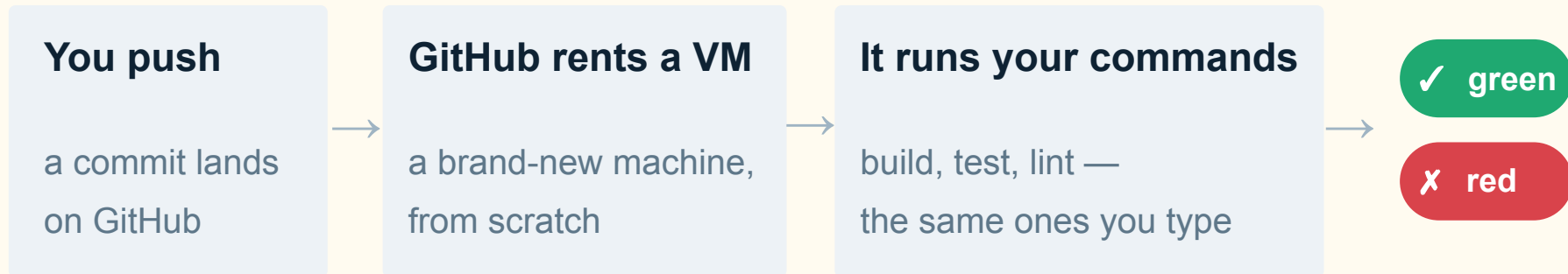


The ROS part of our CI pipeline



Github Actions

You push. A stranger's computer checks your work.



...and then the machine is destroyed.

The machine is thrown away

Nothing you install persists. That is the feature — it is why CI catches "I forgot to commit that file".

Exit code 0 is the whole contract

CI does not understand your tests. It runs a command and looks at the exit code. 0 is green. Anything else is red.

Vocabulary of Github Actions CI

- 1 Event** Something happened in the repo — usually a push or a pull request.
- 2 Workflow** A YAML file in `.github/workflows/` that says "when X happens, do Y".
- 3 Job** A chunk of work that gets its own VM. Jobs run in parallel.
- 4 Step** One command inside a job. Each step runs within a new shell.

The same four words, in a file

on:

push:

jobs:

say-hello:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v7
- run: echo "Hello from \$(hostname) "

on: the EVENT that wakes it up

jobs: named by you; own fresh VM

runs-on: the machine GitHub rents you

uses: someone else's reusable action

run: your own shell command

Github Actions CI Outcomes

✓ SUCCESS

Every command exited with 0.
Safe to merge.

✗ FAILURE

Something exited non-zero. Click through to the step, read the error messages.

Who pays for this?

Free for public repos. Private repos get a monthly minute allowance per org — Linux minutes are the cheap ones.

Can it run on our machine?

Yes — a self-hosted runner. That is how you would test against real hardware.

DIY

<https://github.com/patrick7166/ros2-ci-workshop2026>

Fork this repo to try github actions CI

Trigger Github Actions

```
$ echo "# workshop $(date +%H:%M)" >>  
README.md  
  
$ git commit -am "trigger CI"  
  
$ git push
```

Then open the Actions tab and refresh

Github Actions CI ROS 2 pipeline

```
rosdep install → colcon build → colcon test
```

02-ros2-build-test.yml

Where ROS comes from

Docker Container:

```
container:  
  image: ros:jazzy-ros-base
```

The runner is a bare Ubuntu box with no ROS on it.

We can run our steps inside the official ROS image, which already has colcon and rosdep. Same image you would use on the robot.

Packages: rosdep

```
rosdep install --from-paths .  
  --ignore-src -y
```

rosdep reads the <depend> tags in package.xml and apt-installs them.

So CI is also a test of whether your package.xml contains all packages you need.

Reminder

1 Every run: block is a new shell — and it may not be bash

2 colcon test exits 0 even when tests fail

colcon test: two things to get right

1 setup.py — tell colcon to use pytest, not unittest

```
tests_require=['pytest'],  
  
# Without it, Python 3.12 unittest exits code 5 with "NO TESTS RAN".  
# Plain def test_*( ) functions are invisible to the unittest runner.
```

2 before colcon test — PYTHONPATH for the pytest subprocess

```
source install/setup.bash  
  
colcon test --packages-select turtle_guard --event-handlers console_direct+  
# Without it, the pytest subprocess has no PYTHONPATH, cannot import your  
# packages
```

Let's try and break it

Is this a bug?

```
- return max(-limit, min(limit, value))  
+ return min(limit, value)
```

Would you catch that in code review at 6pm on a Friday?

Forward speed is still limited.

It looks fine if you drive it forward once.

Reverse is now unlimited.

The robot backs into the wall at full speed.

```
$ ./demo/break-it.sh && git commit -am "simplify clamp" && git push
```

A good failure tells you what broke

```
FAILED test/test_safety.py::test_negative_value_is_saturated_symmetrically  
FAILED test/test_safety.py::test_clamp_twist_limits_both_axes_independently  
  
2 failed, 7 passed in 0.04s
```

x 2 checks failed

*It did not say "something broke". It named the behaviour that broke.
That is what a well-named test buys you.*

This is the moment CI is actually for.

Not after you merge — before.

The check runs against the merged result, and you can [make GitHub refuse the merge](#) until it is green.

Advance features

More Advantages of a Pipeline

1 A separate lint job

Runs in parallel with the build. Get style feedback in 40 seconds instead of 6 minutes.

2 A distro matrix

One file, several ROS distros. This is how you find out you have broken the distro you are migrating to, before migration itself..

2. Distro matrix

Matrix: build-and-test

✓ Build & test (jazzy)

36s

✓ Build & test (kilted)

34s

✓ CI passed

3s

1. Parallel lint style check

Matrix: lint

✓ 3 jobs completed

Show all jobs

More Advantages of a Pipeline

3 Artifacts

The test XML and logs are downloadable from the run page regardless of the test outcomes.

Artifacts

Produced during runtime

Name	Size	Digest
 test-results-jazzy	6.83 KB	sha256:072931cbf670746...   
 test-results-kilted	6.81 KB	sha256:b4600575b86ed58...   



More Advantages of a Pipeline

4 Concurrency

Push twice in a minute and the first run is cancelled instead of queueing.

Saves minutes and patience.

```
20 # One run per branch. A new push cancels the previous, still-running one.
21 concurrency:
22     group: ${github.workflow}-${github.ref}
23     cancel-in-progress: true
```

Questions?

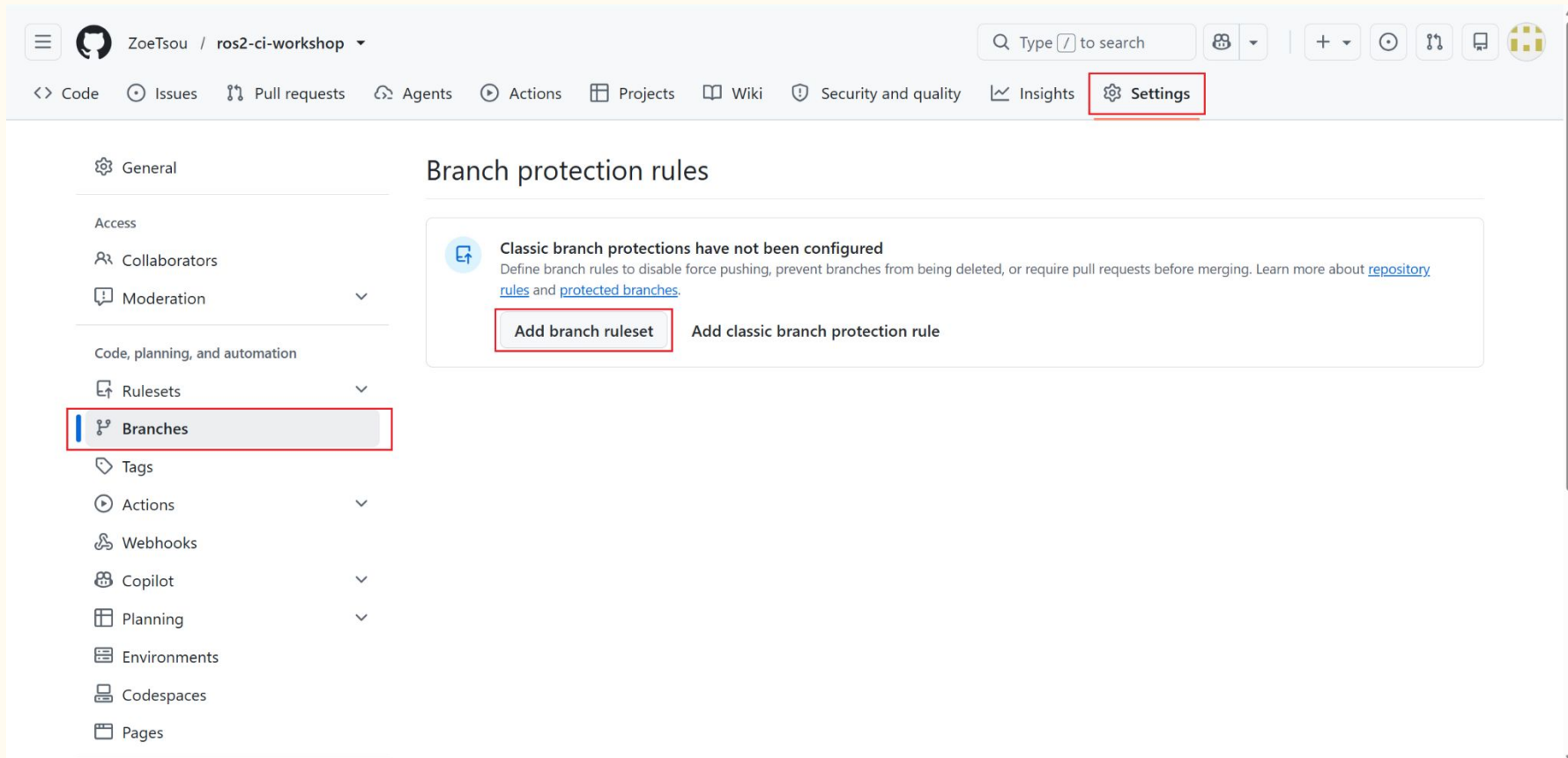
Appendix



What is actions/checkout@v7?

- Repo: actions/checkout
- Version tag: v7
- What it does: downloads your repository's code into the GitHub Actions runner so the later steps can actually access it.

Until you turn this on, CI is a suggestion



The screenshot shows the GitHub repository settings for 'ZoeTsou / ros2-ci-workshop'. The 'Settings' tab is selected in the top navigation bar. In the left sidebar, the 'Branches' option is highlighted with a red box. The main content area is titled 'Branch protection rules' and displays a message: 'Classic branch protections have not been configured'. Below this message, there are two buttons: 'Add branch ruleset' (highlighted with a red box) and 'Add classic branch protection rule'. The message text indicates that branch rules can be used to disable force pushing, prevent branch deletion, or require pull requests before merging, with links to 'repository rules' and 'protected branches' for more information.

Settings

General

Access

Collaborators

Moderation

Code, planning, and automation

Rulesets

Branches

Tags

Actions

Webhooks

Copilot

Planning

Environments

Codespaces

Pages

Branch protection rules

Classic branch protections have not been configured

Define branch rules to disable force pushing, prevent branches from being deleted, or require pull requests before merging. Learn more about [repository rules](#) and [protected branches](#).

Add branch ruleset Add classic branch protection rule

Until you turn this on, CI is a suggestion

The screenshot shows the GitHub interface for creating a new branch ruleset. The left sidebar contains navigation links: Code, Issues, Pull requests, Agents, Actions, Projects, Wiki, Security and quality, Insights, and Settings (highlighted). The main content area is titled 'Rulesets / New branch ruleset'. It includes a section 'Protect your most important branches' with a description of rulesets. Below this, the 'Rule Name' is set to 'Require all CI tests passed' and the 'Enforcement status' is 'Disabled'. The 'Bypass list' section is currently empty, with a red arrow pointing to the 'Actors' tab. The text 'Bypass list is empty' is displayed at the bottom of the screenshot.

General

Access

Collaborators

Moderation

Code, planning, and automation

Rulesets

Rulesets

Branches

Tags

Actions

Webhooks

Copilot

Planning

Environments

Codespaces

Rulesets / New branch ruleset

Protect your most important branches

Rulesets define whether collaborators can delete or force push and set requirements for any pushes, such as passing status checks or a linear commit history.

Rule Name *

Require all CI tests passed

Enforcement status

Disabled

Bypass list

Exempt roles, teams, agents, apps, and users from this ruleset by adding them to the bypass list.

Actors

All Roles Teams Apps Users Others

+ Add bypass

Bypass list is empty

Until you turn this on, CI is a suggestion

☐ **Require deployments to succeed**

Choose which environments must be successfully deployed to before refs can be pushed into a ref that matches this rule.

☐ **Require signed commits**

Commits pushed to matching refs must have verified signatures.

☐ **Require a pull request before merging**

Require all commits be made to a non-target branch and submitted via a pull request before they can be merged.

☒ **Require status checks to pass**

Choose which status checks must pass before the ref is updated. When enabled, commits must first be pushed to another ref where the checks pass.

Hide additional settings ^

☐ **Require branches to be up to date before merging**

Whether pull requests targeting a matching branch must be tested with the latest code. This setting will not take effect unless at least one status check is enabled.

☐ **Do not require status checks on creation**

Allow repositories and branches to be created if a check would otherwise prohibit it.

No required checks

+ Add checks ▾

No checks have been added

[Learn more about status checks](#)

Add Checks

Q CI

Group newEntries

☐ + Add CI Any source

Suggestions

☒ **CI passed** GitHub Actions

☒ **Block force pushes**

Prevent users with push access from force pushing to refs.

☐ **Require code scanning results**

Choose which tools must provide code scanning results before the reference is updated. When configured, code scanning must be enabled and have results for both the commit and the reference being updated.